```
quadIndex.visit(function(node,x1,y1,x2,y2) {
  if (node.point) {
    if (node.point.x >= e[0][0] && node.point.x <= e[1][0]
        &&node.point.y >= e[0][1] && node.point.y <= e[1][1]) {
      node.point.selected = true;
    }
  }
  return x1 > e[1][0] || y1 > e[1][1] || x2 < e[0][0] || y2 < e[0][1];
})

d3.selectAll("circle")
    .filter(function(d) {
        return d.selected;
    })
    .style("fill", "darkred");
};
```

Calls .visit()

Checks each point to see if it's inside our brush extent and sets selected to true if it is

Checks each node to see if it's a point or a container

Checks to see if this area of the quadtree falls outside our selection

Shows which points were selected

The results are impressive and much faster. In figure 11.14, I increased the number of points to 10,000 and still got good performance. (But if you're dealing with datasets that large, I recommend switching to canvas, because forcing the browser to manage all those SVG elements is going to slow things down.)

How does it work? When you run the `visit` function, you get access to each node in the quadtree, from the most generalized to the more specific. With each node, which we access in listing 11.16 as `node`, you also get the bounds of that node (`x1`, `y1`, `x2`, `y2`). Because nodes in a quadtree can either be the bounding areas or the actual points that generated the quadtree, you have to test if the node is a point and, if it is, you can then test if it's in your brush bounds like we did in our earlier example. The final piece of the `visit` function is where it gets its power, but it's also the most difficult to follow, as you can see in figure 11.15.

The `visit` function looks at every node in a quadtree, unless `visit` returns `true`, in which case it stops searching that particular quadrant and all its child nodes. So you test to see if the node you're looking at (represented as the bounds `x1,y1,x2,y2`) is entirely outside the bounds of your selection area (represented as the bounds `e[0][0]`, `e[0][1]`, `e[1][0]`, `e[1][1]`). You create this test to see if the top of the selection is below the bottom of the node's bounds; if the bottom of the selection is above the top of the node's bounds; if the left side of the selection is to the right of the right side of the node's bounds; or if the right side of the selection is to the left of the left side of the node's bounds. That may seem a bit hard to follow (and sure takes up more time as a sentence than it does as a piece of code), but that's how it works.

```
return x1 > e[1][0] || y1 > e[1][1] || x2 < e[0][0] || y2 < e[0][1]
```

Left of node greater than right of selection

Bottom of node greater than top of selection

Right of node less than left of selection

Top of node less than bottom of selection

Figure 11.15 The test to see if a quadtree node is outside a brush selection involves four tests to see if it is above, left, right, or below the selection area. If it passes `true` for any of these tests, then the quadtree will stop searching any child nodes.